

SQLite Programming

Weihai Yu

September 05, 2019

SQLite overview

- open-source
- embedded
 - compact
 - zero-configuration
- portable
- simple
- not complete SQL
 - limited concurrency

Embedded in app processes

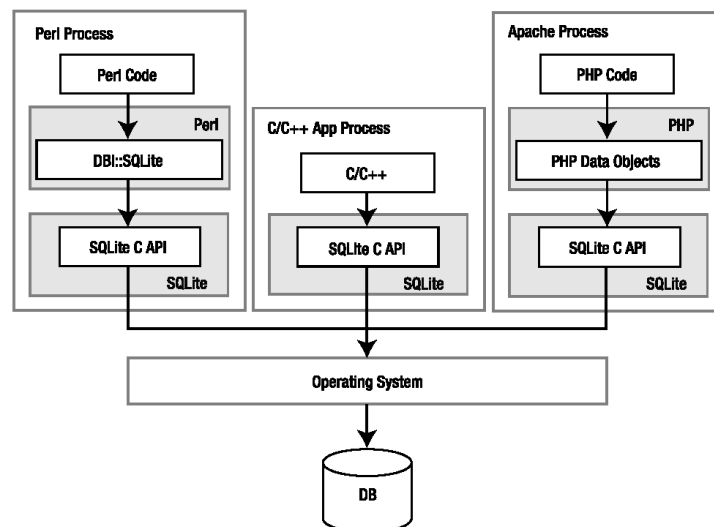


Figure 1-1. SQLite embedded in host processes

- commands starts with a “.”: .help, .quit, .tables, .schema, .mode column, .headers on, .nullvalue null, .dbinfo ...
- sql statements terminates with a “;”: select distinct ...from ...where ...order by ...;

SQLite architecture

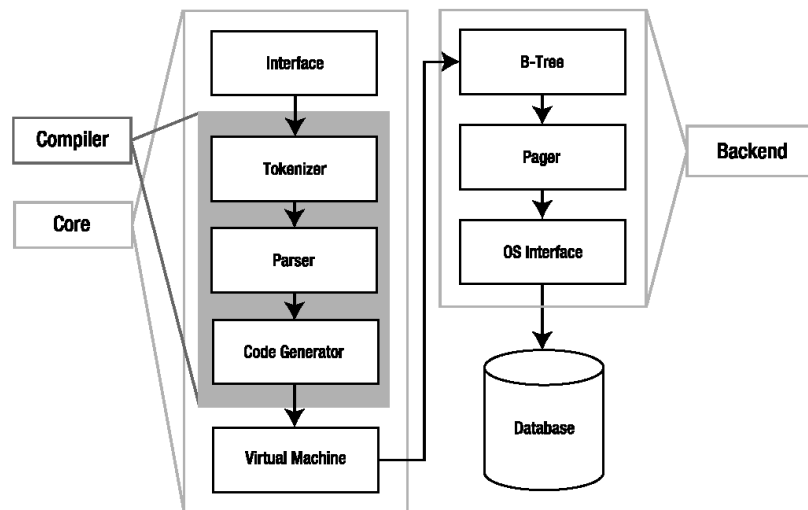


Figure 1-2. SQLite's architecture

C API object model

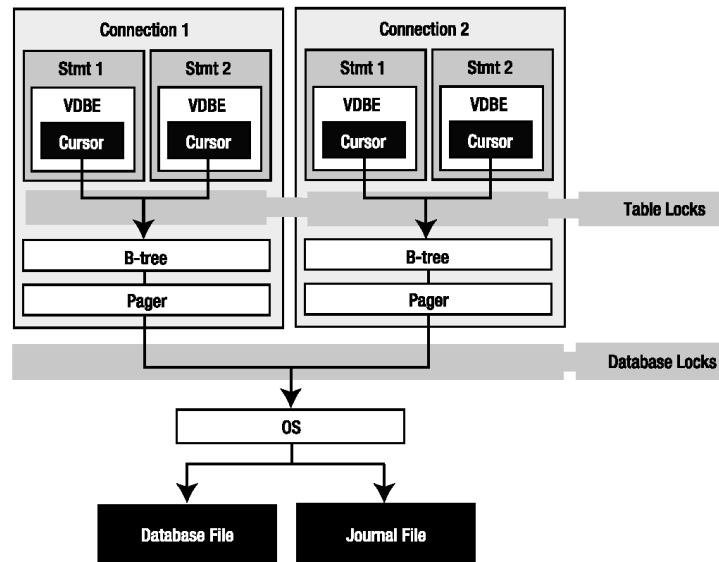


Figure 5-1. *SQLite C API object model*

Statement processing

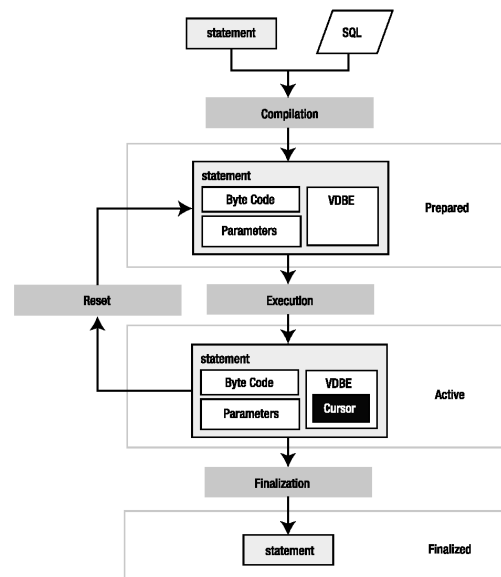


Figure 5-2. *Statement processing*

Prepared query

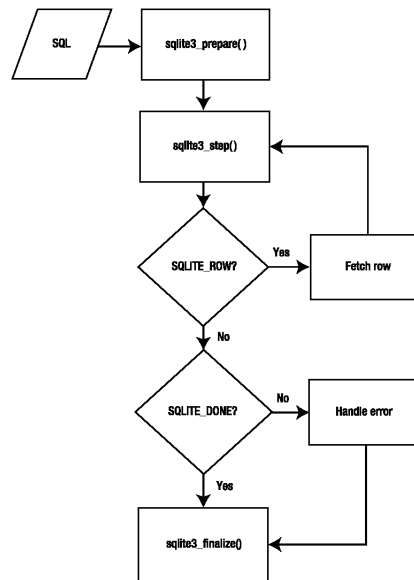


Figure 6-1. Prepared query processing

```

#include <stdio.h>
#include <sqlite3.h>

int main(int argc, char **argv)
{ int rc, i, ncols;
  sqlite3 *db;
  sqlite3_stmt *stmt;
  char *sql;
  const char *tail;
  rc = sqlite3_open(
    "me.sqlite3", &db);
  if(rc) {
    fprintf(stderr, ...);
    sqlite3_close(db);
    exit(1);
  }
  sql = "select * from t1;";

  rc = sqlite3_prepare(
    db, sql, strlen(sql),
    &stmt, &tail);
  if(rc != SQLITE_OK) { ... }

  rc = sqlite3_step(stmt);
  ncols =
    sqlite3_column_count(stmt);

```

```

for(i=0; i < ncols; i++)
    sqlite3_column_xx(stmt, i));

while(rc == SQLITE_ROW) {
    ...
    rc = sqlite3_step(stmt);
}
sqlite3_finalize(stmt);
sqlite3_close(db);
return 0;
}

```

- If the string in sql contains multiple SQL statements, the statements that have not been prepared are stored in **tail**
- xx in `sqlite3_column_xx` could be one of `int`, `double`, `int64`, `blob`, `text` and `text16`

Wrapped query

```

db.exec("INSERT INTO course (course_id, title)
        VALUES ('INF-2700', 'DB')");

tbl = db.get_table("SELECT * FROM course");

for (i = 0; i < tbl.rows; i++)
    for (j = 0; j < tbl.cols; j++)
        tbl[i][j];

```